Relationer mellan objekt

Association, aggregat och komposition

Några viktiga begrepp inom OOP

1. Modellering
   i. Modellera objekt i verkligheten i programmet
   ii. Tillstånd (via datamedlemmar) + operationer (metoder)
2. Inkapsling (encapsulation)
   i. Det som andra klasser inte bör/behöver känna till görs privat
   ii. Andra klasser har endast åtkomst till det publika, klassens gränssnitt (interface)
3. Hög kohesion
   i. Varje typ/koncept i problemområdet modelleras i en egen klass
   ii. En klass abstraherar en och endast en sak

Modellera lösningen

- Vilka objekt och klasser behövs?
- Objekten är beroende av varandra - vilka relationer finns mellan objekten?
  - Vilka andra objekt måste ett givet objekt känna till?
  - Är ett givet objekt uppbyggt av andra objekt (aggregat)?

Kortspelet 21 – hitta kandidater till objekt/klasser

- Hitta kandidater till objekt i nedanstående beskrivning
- *Dealern blandar kortleken och delar ut 2 kort till spelarens hand. Spelaren betraktar korten och beräknar deras totala poäng. Om kortens totala poäng är 21 har spelaren vunnit, om poängen är över 21 har spelaren förlorat. Om poängen är mindre än 21 anger spelaren om denne vill ha fler kort. . .*

Kortspelet 21 – hitta kandidater till objekt/klasser

- *Dealern blandar kortleken och delar ut 2 kort till spelarens hand. Spelaren betraktar korten och beräknar deras totala poäng. Om kortens totala poäng är 21 har spelaren vunnit, om poängen är över 21 har spelaren förlorat. Om poängen är mindre än 21 anger spelaren om hon vill ha fler kort. . .*

- Kandidater till objekt, och därmed klasser, är ofta substantiv i beskrivningen …
- … men vi måste naturligtvis rensa bland kandidaterna och endast behålla det som är relevant för applikationen

Relationer mellan objekt i UML

- Bra med en grafisk notation för att visualisera objekten i modellen och relationer mellan dessa
  - ”Mind map”/skiss – före implementationen
- Unified Modeling Language
  - Oberoende av programmeringsspråk
- Kortvarig relation: beroende, dependency



- Långvariga relationer
  - Mer eller mindre objektens livstid
- Association: känner till



- Aggregat: har, består/uppbyggd av
  - Delat aggregat (shared aggregation)

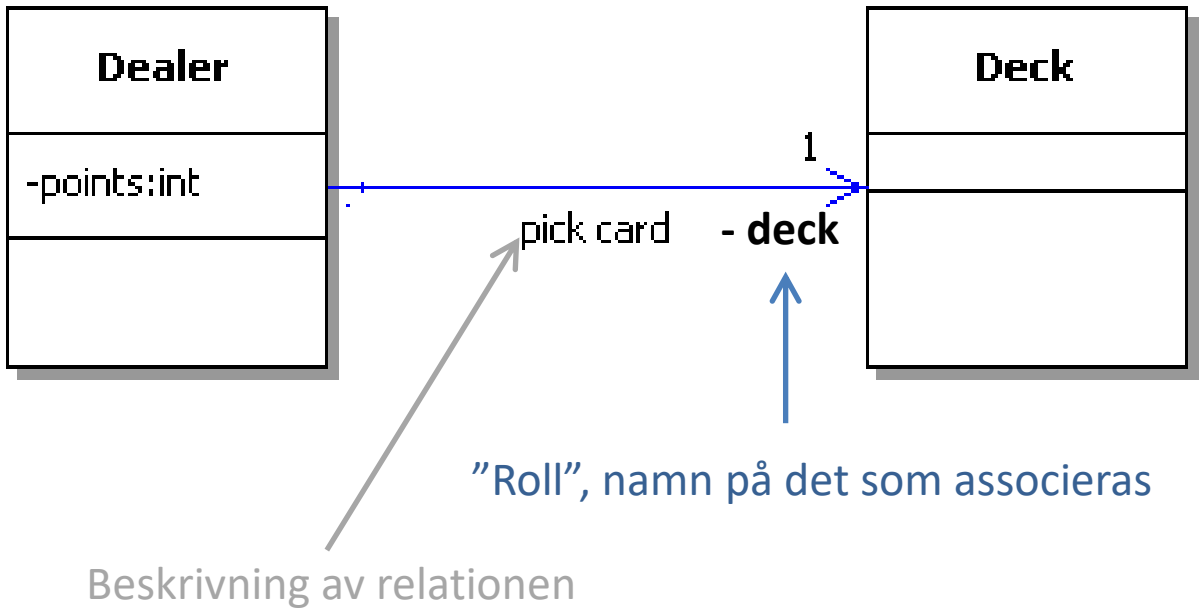


  - Komposition (composite aggregation)



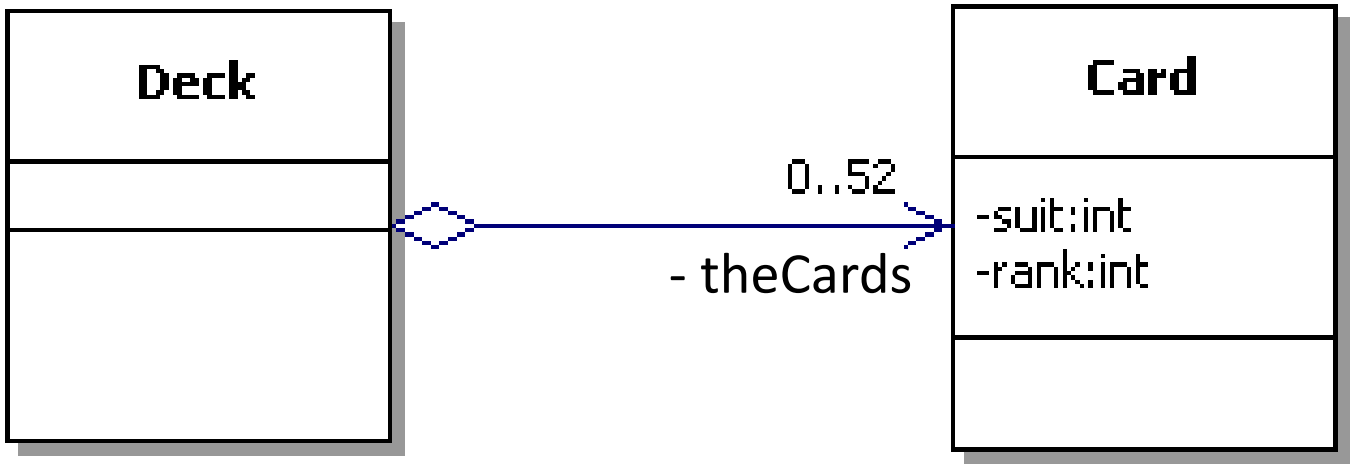
Association, känner till

- Dealern *känner till* kortleken
- Dealern *känner till* spelaren (delar ut kort)



(Delat) aggregat, har eller består av

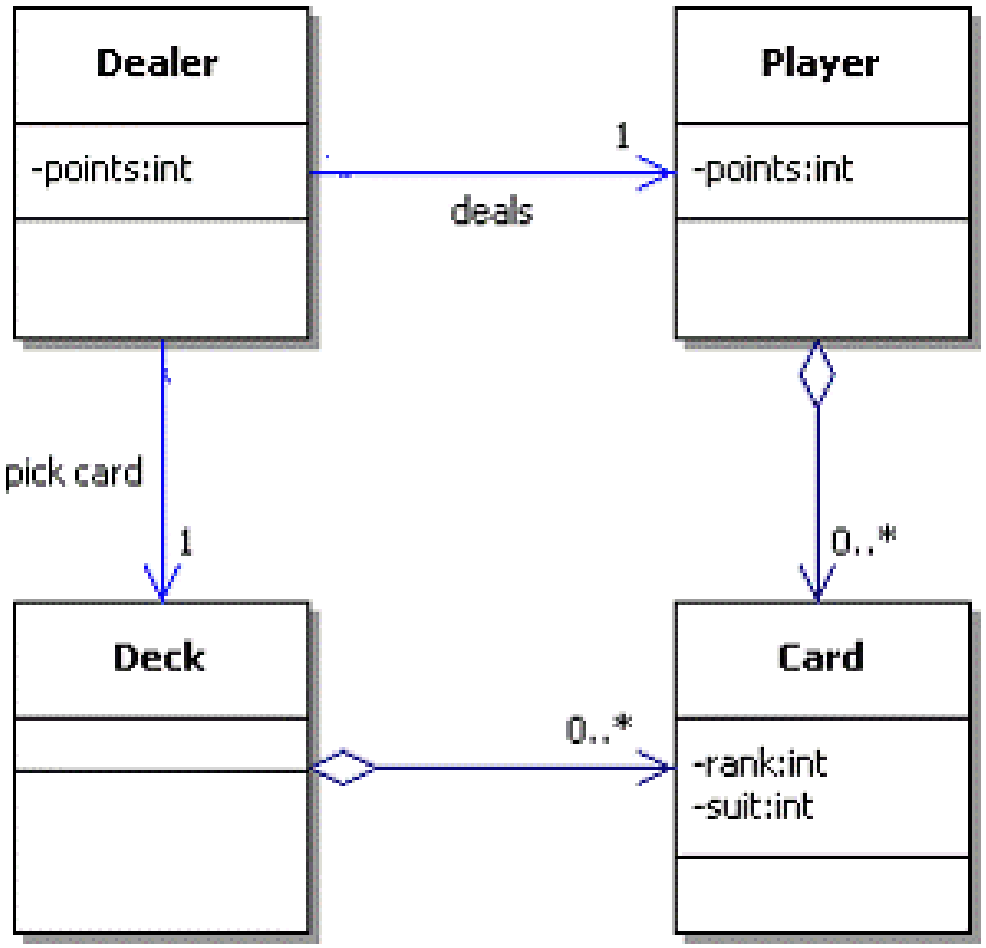
- Kortleken *har, består av*, 0 till 52 kort
- Spelaren *har* ett antal kort – association eller aggregat



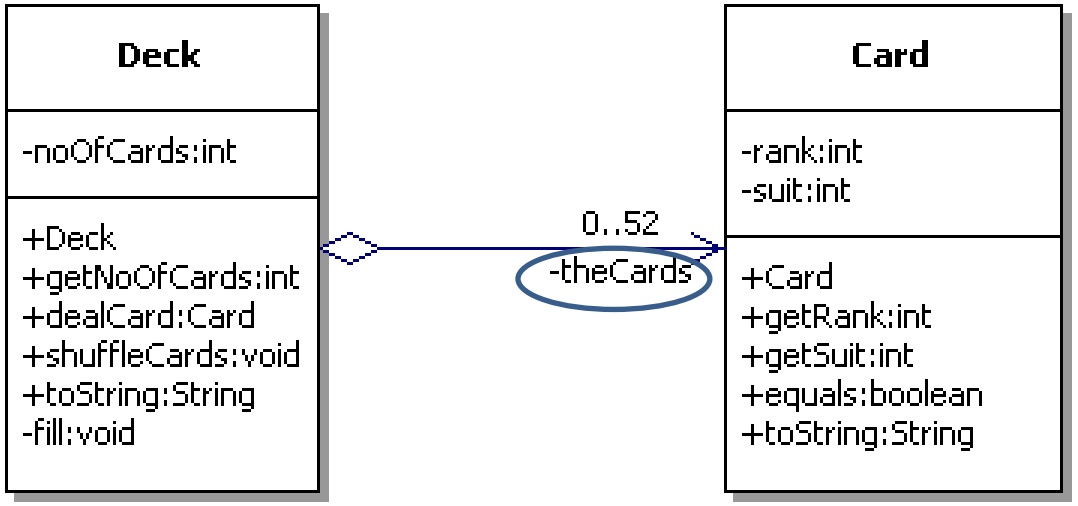
Aggregatet –
det sammansatta

Klassdiagram med objektrelationer för kortspelet 21

- Unified Modelling Language, UML
- I detta fall en *domänmodell*, en beskrivning av ”verkligheten”



Implementation av association och aggregat



- Association och aggregat realiseras i koden med en datamedlem, referens, i aggregerande klassen
- Om flera objekt, i samma roll, aggregeras eller associeras, används en array eller en lista

- Rollnamnet, dvs namnet på motsvarande datamedlem, anges i änden på relationen

Aggregat, implementation

```
public class Deck {

    private Card[] theCards; // deck is an aggregate of cards
    private int noOfCards;

    // create a new deck with 52 cards.
    public Deck() {
        theCards = new Card[52];
        int noOfCards = 0;
        // create the 52 individual card objects
        for (Suit suit : Suit.values()) {
            for (Rank rank  : Rank.values()) {
                theCards[noOfCards ++] = new Card(suit, rank);
            }
        }
    }
```

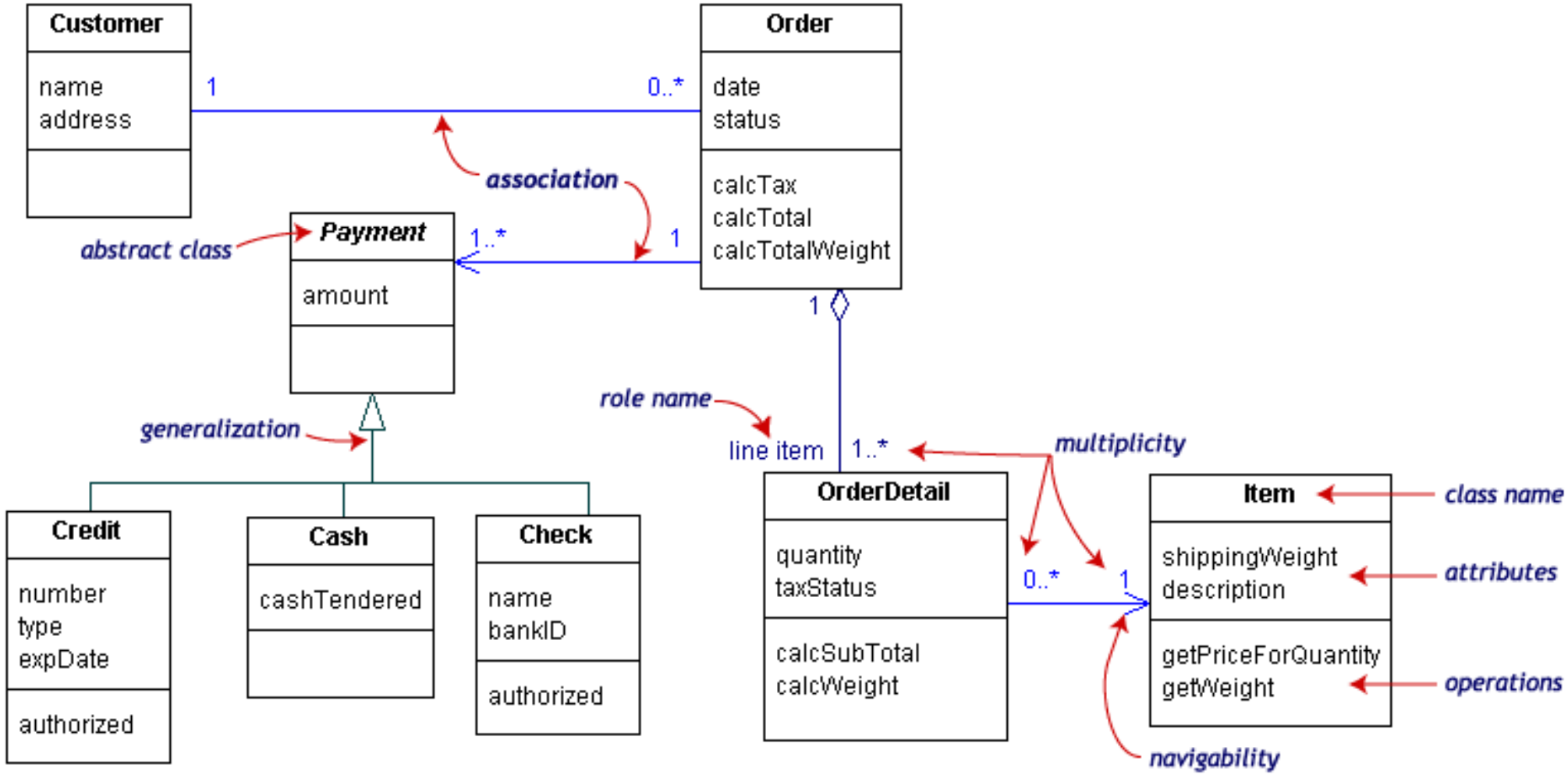
Aggregat, bättre implementation med lista

```
public class Deck {

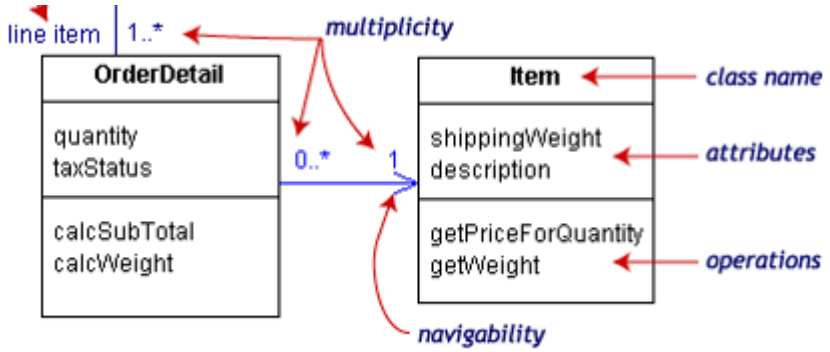
    private ArrayList<Card> theCards; // deck is an aggregate of cards

    // create a new deck with 52 cards.
    public Deck() {
        theCards = new ArrayList<>();
        // create the 52 individual card objects
        for (Suit suit : Suit.values()) {
            for (Rank rank  : Rank.values()) {
                theCards.add(new Card(suit, rank));
            }
        }
    }
```

Komplexa modeller – begripligare nu?



Exempel på association




```
public class OrderDetail {

    private final Item item;
    private int quantity;

    public OrderDetail(Item item, int quantity) {
        this.item = item;
        this.quantity = quantity;
    }

    public Item getItem() {
        return item;
    }

   …
}
```

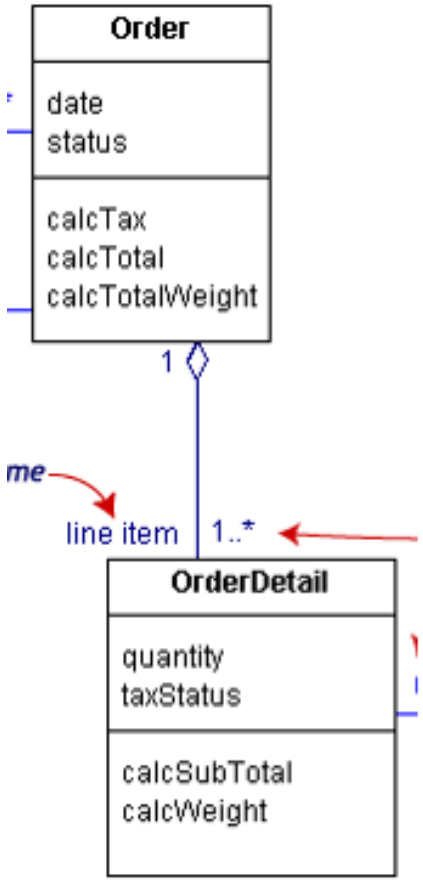
```
public class Item {

    private final String name;
    private final double price;

    public Item(String name, double price) {
        this.name = name;
        this.price = price;
    }

    …
}
```

Exempel på aggregat




```
public class Order {

    private final String customer;
    private final ArrayList<OrderDetail> orderDetails;

    public Order(String customer) {
        this.customer = customer;
        this.orderDetails = new ArrayList<>();
    }
    …

    public void addOrderDetail(Item item, int quantity) {
        orderDetails.add(new OrderDetail(item, quantity));
    }

    public ArrayList<OrderDetail> getOrderDetails() {
        return new ArrayList<OrderDetail>(orderDetails); // copy
    }
    …
}
```

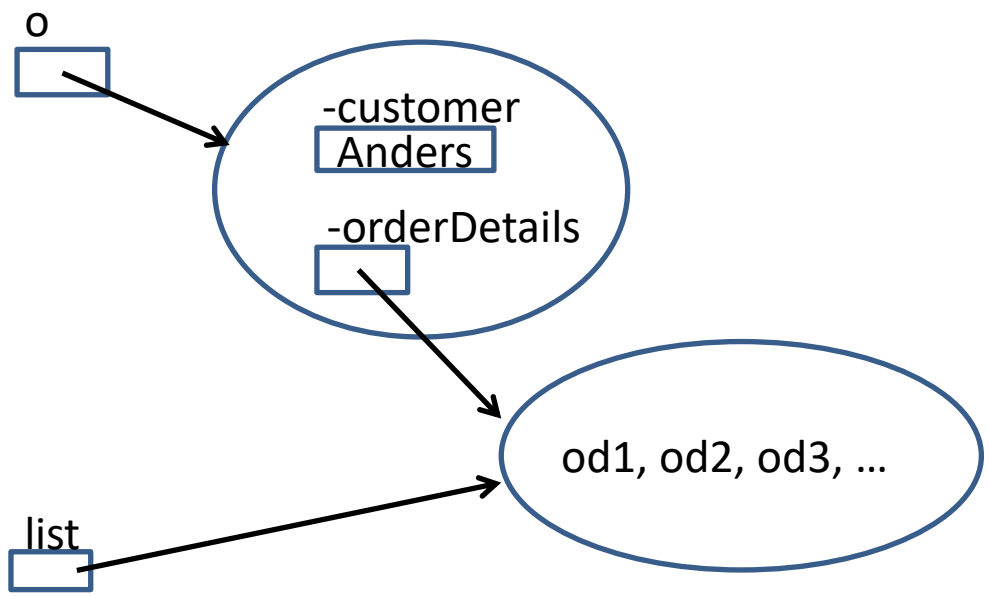
Varför returnerar vi en kopia av listobjektet?

```
Order o = new Order(”Anders”);
o.addOrderDetail(…);
```

Datamedlemmen theAdress är en referens till ett Address-objekt

```
ArrayList<OrderDetails> list =
    o.getOrderDetails();
```

Om metoden *inte* skapar en kopia av objektet får vi en referens till data som är deklarerat privat(!)

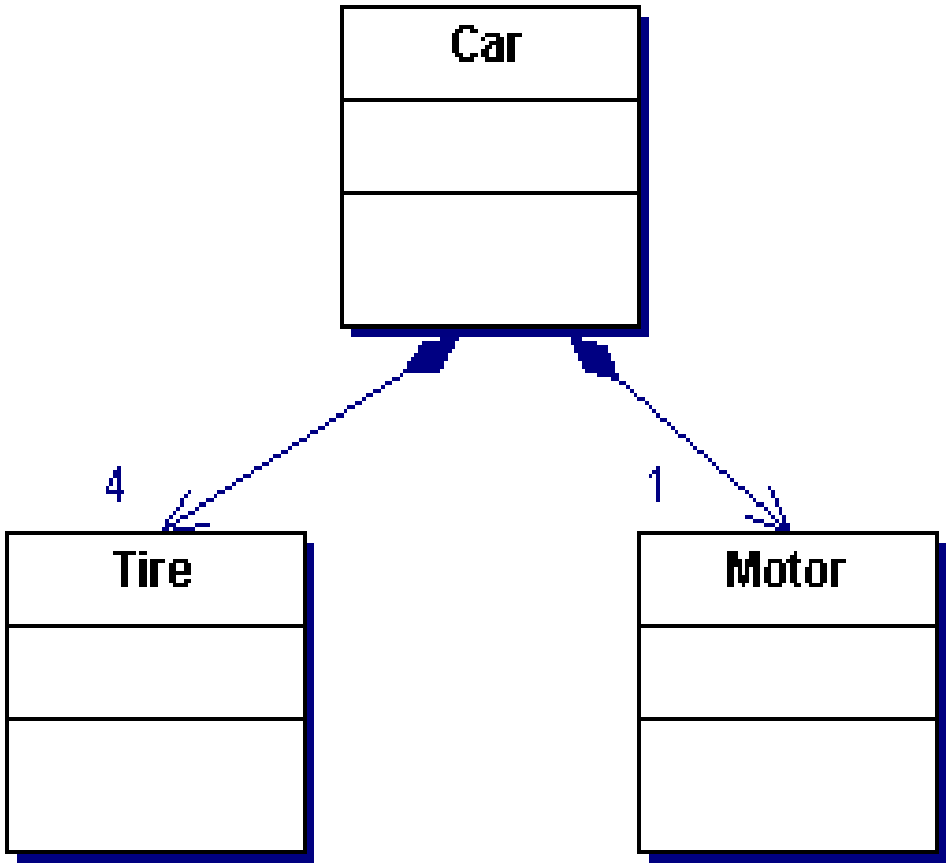


Datamedlemmar som refererar objekt

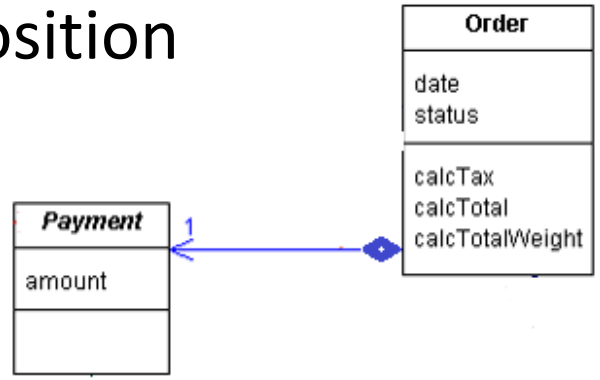
- Satser som ”x = y” och ”return x” innebär i Java alltid en kopiering av variabelns ”värde”
  - Primitivt data: ”värdet” är just värdet, ex. 42
  - Referensvariabler, referenser till objekt: värdet är en ”adress”
    - Vid kopiering är det alltså adressen/referensen som kopieras, inte det refererade objektet
- Access-metoder riskerar alltså att lämna ut en referens till ett privat objekt – eventuellt möjligt att ändra i privat data
  - Om det inte är avsikten, returnera istället en referens till en kopia av objektet
  - Använd new, clone() eller liknande
- Om objektet som refereras är av ”immutable” typ behöver vi inte skapa en kopia
  - Därför kopierar vi aldrig String-objekt i access-metoder

Komposition, composite aggregation

- Delobjekten helt interna
  - får ej delas med andra aggregat, dvs …
  - … aggregerande klass får inte lämna ut referenser till interna objekt – skapa kopior av objektet istället
  - när aggregatet ”dör”, dör även delobjekten



Exempel på komposition




```
public class Payment {

    private final Order order;
    private Status status;

    public Payment(Order order) {
        this.order = order;
        this.status = Status.Created;
    }
    …

    public double getTotal() {
        return order.getTotal();
    }
    …
}
```

```
public class Order {

    private final String customer;
    private final Payment payment;

    public Order(String customer) {
        …
        this.payment = new Payment(this);
    }
    …

    public Payment copyOfPayment() {
        Payment copy = new Payment(this);
        copy.setStatus(payment.getStatus());
        return copy;
    }
    …

    public void setPaymentStatus(Status status) {
        payment.setStatus(status);
    }
    …
}
```

Andra typer av objekt än ”ting”

- *Dealern blandar kortleken och delar ut 2 kort till spelarens hand. Spelaren betraktar korten och beräknar deras totala poäng. Om kortens totala poäng är 21 har spelaren vunnit, om poängen är över 21 har spelaren förlorat. Om poängen är mindre än 21 anger spelaren om denne vill ha fler kort. . .*
- *Poängen beräknas enligt regeln…*
- *Dealern delar sedan kort till sig själv enligt reglerna…*
- *Spelet fortsätter sedan med nästa spelare i turordningen…*

”Abstrakta” ting som objekt

- Logik, regler, …
- Händelser, t.ex. information att användaren agerat och hur
- Enbart data som ska skickas till annat objekt ex.
  - en lista med kort för utskrift
  - information om en transaktion i en bankapplikation
- Abstraktioner av protokoll, nätverksuppkopplingar m.m.
- Vissa objekt kanske inte behöver abstraheras i mjukvarumodellen
  - Behöver dealern representeras i vårt (enkla) 21-sepl?

Spelet 21, designmodellen
”Ritningen till applikationen”

